The console.log Trick That Senior Developers Don't Want You to Know

Saurabh Raj

Okay, I'll be honest — senior developers absolutely want you to know this. In fact, they've been using it for years, while the rest of us have been cluttering our code with `console.log("here")`, `console.log("here2")`, and the classic

`console.log("aaaaaa").`

These techniques will transform you from someone who debugs by trial and error into a developer who pinpoints issues with surgical precision. Master these, and you'll be the one your team turns to when something breaks in production.

1. The Game-Changer: Computed Property Names

Stop doing this:

```
const user = { name: "Alice", age: 30 };
const product = { id: 123, price: 49.99 };

console.log("user", user);
console.log("product", product);
```

Start doing this:

```
console.log({ user, product });
```

Why this matters: Using ES6 shorthand object syntax wraps your variables in an object, so you instantly see both the variable name AND its value in the console. No more guessing which log corresponds to which variable when you

have 20 of them.

Output:

2. The Secret Weapon: console.table()

When you're dealing with arrays of objects, `console.log` is virtually useless. Try this instead:

```
const users = [
  { name: "Alice", age: 30, role: "Admin" },
  { name: "Bob", age: 25, role: "User" },
  { name: "Charlie", age: 35, role: "Moderator" }
];

console.table(users);
```

This renders a beautiful, sortable table in your browser console. You can click column headers to sort, and it's infinitely more readable than a nested object dump. I've seen this single technique cut debugging time in half for data-heavy applications.

3. The Stack Trace Shortcut: console.trace()

Ever wonder “how did I even get here?” when your function gets called from multiple places?

```
function processPayment(amount) {
    function innerFn() {
        console.trace("Payment processing started");
    }

    innerFn()
}

processPayment(20)
```

`console.trace()` prints the full call stack, showing you the exact path the code took to reach that point. This is absolutely crucial when debugging complex applications where a function might be called from 5 different places.

4. The Conditional Breakpoint: console.assert()

Instead of wrapping your logs in if statements:

```
if (user.age < 18) {
  console.log("Underage user detected!");
}
```

Use this:

```
console.assert(user.age >= 18, "Underage user detect
```

It only logs when the assertion fails (when the condition is false). Cleaner code, less noise in your console, and it includes the actual data automatically.

5. The Performance Monitor: console.time()

Stop guessing how long your code takes to run:

```
console.time("API Call");

fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => {
    console.timeEnd("API Call");
    return data;
  });

console.timeEnd("API Call");
```

This tells you exactly how many milliseconds elapsed between `console.time()` and `console.timeEnd()`. I use this constantly for comparing different implementations or finding performance bottlenecks.

Output:

```
API Call: 342.87ms
```

6. The Styled Logger: CSS in Console

Make your important logs impossible to miss:

```
console.log(
  "%c🚨 CRITICAL ERROR",
  "color: red; font-size: 20px; font-weight: bold; b
);
```

You can add CSS styling to console logs using `%c`. This is perfect for:

- Error states that need immediate attention
- Success messages in development
- Separating sections of complex debugging output

7. The Group Organizer: console.group()

When you have related logs, group them:

```
console.group("User Authentication");
```

```
console.log("Checking credentials...");
console.log("Token:", token);
console.log("Validating...");
console.groupEnd();

console.group("API Response");
console.log("Status:", response.status);
console.log("Data:", response.data);
console.groupEnd();
```

This creates collapsible groups in your console, making it dramatically easier to navigate through lots of debug output. Use `console.groupCollapsed()` if you want groups collapsed by default.

8. The Object Deep Dive: console.dir()

For DOM elements or objects with special properties:

```
const element = document.querySelector("#myButton");

console.log(element);  // Shows HTML representation
console.dir(element);  // Shows object properties
```

`console.dir()` displays an interactive list of object properties, which is especially useful for DOM elements where you want to inspect methods and properties rather than the HTML structure.

9. The Log Level Hierarchy: Using the Right Tool

Stop using `console.log()` for everything. JavaScript gives you different log levels for a reason:

```
console.log("Regular information");      // General
console.info("ℹ️ User logged in");       // Informa
console.warn("⚠️ API rate limit at 80%"); // Warnin
console.error("❌ Payment failed");       // Errors
console.debug("🔍 Variable state:", x);   // Debug
```

Why this matters: Modern browser DevTools let you filter by log level. When debugging in production, you can hide all `console.log` and `console.debug` statements and only see warnings and errors. This makes critical issues impossible to miss in a sea of debug output.

10. The DevTools Secret Weapons: 0, $, and More

These magical shortcuts only work in browser DevTools console, but they're absolute game-changers:

$0 — The Last Selected Element

Click on any element in the Elements tab, then switch to Console:

```
$0 // References the currently selected element
$0.style.backgroundColor = 'red';
```

```
$0.classList.add('highlight');
```

Even better: `$1`, `$2`, `$3`, `$4` references the last 5 selected elements. Perfect for comparing or manipulating multiple elements quickly.

$$ — The Superior Selector

Forget `document.querySelectorAll()`:

```
// Instead of this:
Array.from(document.querySelectorAll('.product-card'
  .forEach(card => console.log(card.textContent));

// Use this:
$$('.product-card').forEach(card => console.log(carc
```

`$$()` returns a real array (not a NodeList), so you get all array methods immediately. No more `Array.from()` wrapper.

$ — Quick querySelector

```
$('#header')            // document.querySelector('#h
$('.btn-primary')       // document.querySelector('.b
```

Note: This only works if jQuery isn't loaded. If jQuery is present, `$` is jQuery.

$_ — The Last Result

```
2 + 2
// 4

$_ * 10
// 40
```

`$_` stores the result of the last expression evaluated in the console. Incredibly useful for chaining operations without re-typing.

copy() — Instant Clipboard

```
const data = { users: [...], products: [...] };
copy(data);
// Copied to clipboard as formatted JSON!
```

No more right-clicking and “Copy object” or trying to stringify and select text. Just `copy(anything)` and it's on your clipboard.

getEventListeners() — Event Inspector

```
getEventListeners($0)
// Shows ALL event listeners attached to the selecte
```

```
// Example output:
// {
//   click: [{ listener: ƒ, useCapture: false, pass
//   mouseenter: [{ listener: ƒ, useCapture: false,
// }
```

This is invaluable for debugging event handler issues or finding memory leaks from unremoved listeners.

The Bonus: Live Expression Watch (Chrome DevTools)

This isn't technically a console method, but it's game-changing. In Chrome DevTools, you can create "Live Expressions" that constantly update as your code runs.

1. Open Chrome DevTools
2. Click the "eye" icon in the Console tab
3. Type any JavaScript expression: `user.isAuthenticated`, `cart.items.length`, etc.

It stays pinned at the top of your console and updates in real-time. Perfect for watching state changes without flooding your console.

Conclusion

I spent years thinking `console.log()` was just for printing text. These techniques transformed debugging from a frustrating guessing game into a precise, efficient process.

The real secret senior developers know? **The right debugging technique saves you hours of trial and error.** Master these tools, and you'll spend less time adding logs and more time actually fixing bugs.

Next time you're debugging, try replacing all your plain `console.log` statements with these techniques. Your future self (and your code reviewers) will thank you.

What's your favorite console trick? Drop it in the comments — I'm always looking to level up my debugging game.